

zadania lab 4

Zadanie 1

Napisz funkcję zwracającą wyznacznik macierzy kwadratowej dowolnego rozmiaru.

Jak rozumieć to zadanie?

Wyznacznik można policzyć tylko dla **macierzy kwadratowej**, czyli takiej, która ma tyle samo wierszy co kolumn.

Dla małych macierzy są proste wzory:

Dla macierzy 1×1

$$\det(A) = a_{11}$$

Dla macierzy 2×2

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

Dla większych macierzy

Stosujemy **rozwinięcie Laplace'a** względem pierwszego wiersza:

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} M_{1j}$$

gdzie M_{1j} to minor, czyli wyznacznik macierzy powstałej po skreśleniu pierwszego wiersza i j -tej kolumny.

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz, czy macierz jest kwadratowa.

Krok 2

Jeśli macierz ma rozmiar 1×1 , zwracasz jedyny element.

Krok 3

Jeśli macierz ma rozmiar 2×2 , liczysz wyznacznik ze wzoru:

$$ad - bc$$

Krok 4

Dla większych macierzy:

- tworzysz minory,
- liczysz ich wyznaczniki rekurencyjnie,
- sumujesz wszystko ze znakami $+$ i $-$.

```
import math

def minor(macierz, usun_wiersz, usun_kolumne):
    wynik = []

    for i in range(len(macierz)):
        if i == usun_wiersz:
            continue

        nowy_wiersz = []
        for j in range(len(macierz[i])):
            if j == usun_kolumne:
                continue
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def wyznacznik_macierzy(macierz):
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie jest kwadratowa")

    if n == 1:
        return macierz[0][0]

    if n == 2:
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0]

    det = 0
    for j in range(n):
        podmacierz = minor(macierz, 0, j)
        det += math.pow((-1), 0+j) * macierz[0][j] * wyznacznik_macierzy(podmacierz)

    return det

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]
```

```
print("Wyznacznik macierzy: ", wyznacznik_macierzy(macierz))
```

Wynik

Dla macierzy

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

otrzymujemy:

$$\det(A) = 66$$

Zadanie 2

Napisz funkcję zwracającą transpozycję macierzy dowolnego rozmiaru.

Co to jest transpozycja?

Transpozycja macierzy polega na zamianie:

- wierszy na kolumny,
- kolumn na wiersze.

Jeśli:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

to:

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz liczbę wierszy i kolumn.

Krok 2

Tworzysz nową macierz wynikową.

Krok 3

Dla każdej kolumny starej macierzy tworzysz nowy wiersz.

Czyli element:

$$a_{ij}$$

staje się elementem:

$$a_{ji}$$

Kod

PYTHON

```
def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []
        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])
        wynik.append(nowy_wiersz)

    return wynik

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

print("Przed transpozycją macierzy:")
for wiersz in macierz:
    print(wiersz)

wynik = transpozycja(macierz)

print("Po transpozycji macierzy:")
for wiersz in wynik:
    print(wiersz)
```

Wynik

Dla macierzy:

$$\begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

transpozycja ma postać:

$$\begin{bmatrix} 2 & 0 & -3 \\ 4 & 2 & 3 \\ 6 & -1 & 3 \end{bmatrix}$$

Zadanie 3

Napisz funkcję znajdującą macierz odwrotną do macierzy kwadratowej dowolnego rozmiaru za pomocą:

- a) rozwinięcia Laplace'a
- b) metody Gaussa-Jordana

Zadanie 3a — macierz odwrotna metodą Laplace'a

Idea metody

Jeśli macierz A ma wyznacznik różny od zera, to macierz odwrotna istnieje i można ją policzyć ze wzoru:

$$A^{-1} = \frac{1}{\det(A)} \cdot Adj$$

gdzie:

- $\det(A)$ to wyznacznik,
- Adj to macierz dołączona, czyli transpozycja macierzy dopełnień algebraicznych.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz wyznacznik macierzy.

Krok 2

Jeśli wyznacznik jest równy 0, macierz nie ma odwrotności.

Krok 3

Dla każdego elementu liczysz minor i dopełnienie algebraiczne:

$$C_{ij} = (-1)^{i+j} \det(M_{ij})$$

Krok 4

Tworzysz macierz dopełnień algebraicznych.

Krok 5

Robisz jej transpozycję, czyli macierz dołączoną.

Krok 6

Dzielisz każdy element przez wyznacznik.

```
import math

def minor(macierz, usun_wiersz, usun_kolumne):
    wynik = []

    for i in range(len(macierz)):
        if i == usun_wiersz:
            continue

        nowy_wiersz = []
        for j in range(len(macierz[i])):
            if j == usun_kolumne:
                continue
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def wyznacznik_macierzy(macierz):
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie jest kwadratowa")

    if n == 1:
        return macierz[0][0]

    if n == 2:
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0]

    det = 0
    for j in range(n):
        podmacierz = minor(macierz, 0, j)
        det += math.pow((-1), 0+j) * macierz[0][j] * wyznacznik_macierzy(podmacierz)

    return det

def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []
```



```

    for j in range(liczba_kolumn):
        nowy_wiersz = []
        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])
        wynik.append(nowy_wiersz)

    return wynik

def zeros(n,m):
    macierz = []

    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0)
        macierz.append(wiersz)

    return macierz

def macierz_odwrotna_Laplace(macierz): # punkt a
    n = len(macierz)
    d = wyznacznik_macierzy(macierz)

    if d == 0:
        raise ValueError("Macierz jest osobliwa, nie ma odwrotności")

    C = zeros(n, n)

    for i in range(n):
        for j in range(n):
            M = minor(macierz, i, j)
            C[i][j] = math.pow(-1, i+j) * wyznacznik_macierzy(M)

    Adj = transpozycja(C)

    wynik = zeros(n, n)
    for i in range(n):
        for j in range(n):
            wynik[i][j] = Adj[i][j] / d

    return wynik

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

wynik_Laplace = macierz_odwrotna_Laplace(macierz)

```

```
print("Macierz odwrotna Laplace:")
for wiersz in wynik_Laplace:
    print(wiersz)
```

Wynik

Dla macierzy:

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

otrzymujemy macierz odwrotną:

$$A^{-1} = \begin{bmatrix} 0.13636363636363635 & 0.09090909090909091 & -0.24242424242424243 \\ 0.045454545454545456 & 0.36363636363636365 & 0.030303030303030304 \\ 0.09090909090909091 & -0.2727272727272727 & 0.06060606060606061 \end{bmatrix}$$

Zadanie 3b — macierz odwrotna metodą Gaussa-Jordana

Idea metody

Tworzy się macierz rozszerzoną:

$$[A \mid I]$$

a następnie wykonuje się operacje na wierszach tak, aby lewa część stała się macierzą jednostkową:

$$[I \mid A^{-1}]$$

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz macierz rozszerzoną:

- po lewej masz macierz A ,
- po prawej macierz jednostkową I .

Krok 2

Dla każdej kolumny ustawiasz jedynkę na przekątnej.

Krok 3

Wyzerowujesz pozostałe elementy w tej kolumnie.

Krok 4

Po zakończeniu prawa część jest macierzą odwrotną.

```
import math

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

        # lewa strona: macierz A
        for j in range(n):
            wiersz.append(macierz[i][j])
            # wiersz.append(float(macierz[i][j]))

        # prawa strona: macierz jednostkowa I
        for j in range(n):
            if i == j:
                wiersz.append(1)
            else:
                wiersz.append(0)

        rozszerzona_macierz.append(wiersz)

    # algorytm Gaussa-Jordana
    for i in range(n):
        # jeśli na przekątnej jest 0, zamień wiersze
        if rozszerzona_macierz[i][i] == 0:
            znaleziono = False
            for k in range(i+1, n):
                if rozszerzona_macierz[k][i] != 0:
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =
                    rozszerzona_macierz[k], rozszerzona_macierz[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Macierz nie ma odwrotności")

        # dzielenie całego wiersza przez element główny
        element_glowny = rozszerzona_macierz[i][i]
        for j in range(2 * n):
```

```

        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

# zerowanie pozostałych elementów w tej kolumnie
for k in range(n):
    if k != i:
        wspolczynnik = rozszerzona_macierz[k][i]
        for j in range(2 * n):
            rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j]

odwrotna = []
for i in range(n):
    wiersz = []
    for j in range(n, 2 * n):
        wiersz.append(rozszerzona_macierz[i][j])
    odwrotna.append(wiersz)

return odwrotna

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

wynik_Gauss_Jordan = macierz_odwrotna_Gaussa_Jordana(macierz)

print("Macierz odwrotna Gauss Jordan:")
for wiersz in wynik_Gauss_Jordan:
    print(wiersz)

```

Wynik

Otrzymujemy macierz odwrotną:

$$A^{-1} = \begin{bmatrix} 0.13636363636363635 & 0.09090909090909083 & -0.24242424242424243 \\ 0.045454545454545456 & 0.36363636363636365 & 0.030303030303030304 \\ 0.09090909090909091 & -0.2727272727272727 & 0.06060606060606061 \end{bmatrix}$$

Porównanie wyników obu metod

Obie metody dają tę samą macierz odwrotną.

Mogą pojawić się bardzo małe różnice w zapisie dziesiętnym, ale wynik matematycznie jest ten sam.

Zadanie 4

Napisz funkcję, która wykona mnożenie dwóch macierzy.

Kiedy można mnożyć macierze?

Macierze można mnożyć tylko wtedy, gdy:

- liczba kolumn pierwszej macierzy
- jest równa liczbie wierszy drugiej macierzy.

Jeśli:

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times k}$$

to wynik ma rozmiar:

$$AB \in \mathbb{R}^{m \times k}$$

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz zgodność wymiarów.

Krok 2

Dla każdego elementu wyniku liczysz sumę iloczynów elementów odpowiedniego wiersza i kolumny.

Czyli:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

```
def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz1 = [
    [1, 2],
    [3, 4]
]

macierz2 = [
    [5, 6],
    [7, 8]
]

wynik = mnozenie_macierzy(macierz1, macierz2)

print("Wynik mnożenia:")
for wiersz in wynik:
    print(wiersz)
```

Wynik

Dla:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

otrzymujemy:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Zadanie 5

Korzystając z rozwiązań poprzednich zadań wykonaj następujące mnożenia macierzowe: $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$ i porównaj ich wyniki.

Idea zadania

Jeśli macierz (A) jest odwracalna, to powinno zachodzić:

$$A \cdot A^{-1} = I$$

oraz

$$A^{-1} \cdot A = I$$

gdzie I to macierz jednostkowa.

Czyli zadanie polega na sprawdzeniu, czy wyznaczona wcześniej macierz odwrotna rzeczywiście jest poprawna.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz macierz odwrotną.

Krok 2

Mnożysz:

- $A \cdot A^{-1}$
- $A^{-1} \cdot A$

Krok 3

Porównujesz wyniki z macierzą jednostkową.

Jeśli pojawiają się małe liczby typu:

$$2.220446049250313 \cdot 10^{-16}$$

to traktuje się je jako 0, ponieważ są to błędy zaokrągleń.

```
def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

        # lewa strona: macierz A
        for j in range(n):
            wiersz.append(macierz[i][j])
            # wiersz.append(float(macierz[i][j]))

        # prawa strona: macierz jednostkowa I
        for j in range(n):
            if i == j:
                wiersz.append(1)
            else:
                wiersz.append(0)

        rozszerzona_macierz.append(wiersz)

    # algorytm Gaussa-Jordana
    for i in range(n):
        # jeśli na przekątnej jest 0, zamień wiersze
        if rozszerzona_macierz[i][i] == 0:
            znaleziono = False
            for k in range(i+1, n):
                if rozszerzona_macierz[k][i] != 0:
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =
                    rozszerzona_macierz[k], rozszerzona_macierz[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Macierz nie ma odwrotności")

        # dzielenie całego wiersza przez element główny
        element_glowny = rozszerzona_macierz[i][i]
        for j in range(2 * n):
            rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny
```

```

        # zerowanie pozostałych elementów w tej kolumnie
        for k in range(n):
            if k != i:
                współczynnik = rozszerzona_macierz[k][i]
                for j in range(2 * n):
                    rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
współczynnik * rozszerzona_macierz[i][j]

    odwrotna = []
    for i in range(n):
        wiersz = []
        for j in range(n, 2 * n):
            wiersz.append(rozszerzona_macierz[i][j])
        odwrotna.append(wiersz)

    return odwrotna

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

macierz_odwrotna = macierz_odwrotna_Gaussa_Jordana(macierz)

wynik1 = mnozenie_macierzy(macierz, macierz_odwrotna)
wynik2 = mnozenie_macierzy(macierz_odwrotna, macierz)

```

```
print("Wynik mnożenia A * A^-1:")
for wiersz in wynik1:
    print(wiersz)

print("Wynik mnożenia A^-1 * A:")
for wiersz in wynik2:
    print(wiersz)
```

Wynik

Oba iloczyny powinny być bardzo bliskie macierzy jednostkowej:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

W praktyce może się pojawić na przykład:

$$2.220446049250313e - 16$$

zamiast dokładnego zera. To jest normalne i wynika z ograniczonej dokładności obliczeń zmiennoprzecinkowych.

Wniosek

Wszystkie zadania zostały wykonane poprawnie.

- W zadaniu 1 obliczono wyznacznik macierzy metodą rozwinięcia Laplace'a.
- W zadaniu 2 obliczono transpozycję macierzy.
- W zadaniu 3 wyznaczono macierz odwrotną dwiema metodami: Laplace'a oraz Gaussa-Jordana.
- W zadaniu 4 wykonano mnożenie dwóch macierzy.
- W zadaniu 5 sprawdzono poprawność macierzy odwrotnej przez obliczenie iloczynów $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$.

Otrzymane wyniki są zgodne z teorią, a ewentualne bardzo małe różnice wynikają z błędów zaokrągleń numerycznych.